ESPECIFICACIÓN DE INGENIERÍA Y ARQUITECTURA DE SOFTWARE

ACTIVELO — Manual Técnico

Arquitectura de aplicación Flutter, gestión de estado reactiva (Riverpod), protocolo Bluetooth Low Energy (ESP32), motor de entrenamiento, persistencia en Firebase y entorno de despliegue.

El presente documento describe la arquitectura interna, los componentes de software, los protocolos de comunicación inalámbrica y el modelo de datos de Activelo. Está dirigido a desarrolladores, evaluadores técnicos y mantenedores del sistema que necesiten comprender el funcionamiento integral de la aplicación móvil/web y su interacción con el hardware basado en ESP32.

Índice de contenidos

Sección	Módulo / Capítulo	Alcance técnico
Sección 01	Visión General y Stack Tecnológico	Arquitectura de 3 capas, dependencias principales y soporte multiplataforma.
Sección 02	Arquitectura de Software y Enrutamiento	Estructura de directorios, proveedores Riverpod y mapa de navegación GoRouter.
Sección 03	Protocolo BLE y Comunicación ESP32	UUID de servicio, escaneo, formato de comandos TX, eventos RX y watchdog.
Sección 04	Motor de Entrenamiento y Temporizadores	Ciclo de vida de rondas, escalado RGB, modo bicolor y liberación de recursos.
Sección 05	Subsistema de Audio e Internacionalización	Control de AudioPlayer con filtro anti-rebote y soporte l10n (10 idiomas LTR/RTL).
Sección 06	Autenticación y Modelo de Datos Firebase	Firebase Auth y esquema JSON en Realtime Database para usuarios y sesiones.
Sección 07	Entorno de Desarrollo, Compilación y Debug	Configuración Android SDK, permisos nativos BLE y buenas prácticas de depuración.

01 Visión General y Stack Tecnológico

Activelo implementa una arquitectura desacoplada en tres capas: una interfaz cliente multiplataforma desarrollada en Flutter (Dart), una capa de hardware embebido controlada mediante Bluetooth Low Energy (ESP32 + red de pods LED con sensores) y una capa de servicios en la nube provista por Firebase para autenticación y almacenamiento en tiempo real.

Componente / Paquete	Capa	Responsabilidad técnica en el sistema
`Flutter SDK (Dart)`	Frontend UI	Renderizado nativo en Android y soporte web multiplataforma con diseño responsivo y temas claro/oscuro.
`flutter_riverpod`	Estado	Inyección de dependencias y gestión de estado reactiva mediante StateNotifier, StateProvider y listeners.
`flutter_blue_plus`	Conectividad	Escaneo BLE filtrado por UUID, conexión GATT, escritura en característica RX y suscripción a notificaciones.
`go_router`	Navegación	Enrutamiento declarativo con paso de parámetros tipados (extra) y control de historial.
`firebase_auth`	Seguridad	Autenticación por correo/contraseña, persistencia de sesión y envío de enlaces de restablecimiento.
`firebase_database`	Persistencia	Base de datos NoSQL JSON en tiempo real para perfiles de usuario e historial de métricas.
`audioplayers`	Multimedia	Reproducción de efectos sonoros locales (assets/sounds/) sincronizados con la cuenta regresiva y cierre.

1.1 Adaptación multiplataforma (Android vs. Web)

Mediante la constante de compilación kIsWeb (package:flutter/foundation.dart), la aplicación adapta su comportamiento según la plataforma de ejecución:

- **Ejecución Nativa (Android):** En dispositivos Android nativos, la pantalla BluetoothScreen inicia el escaneo automáticamente en initState(), escucha el stream FlutterBluePlus.isScanning y detecta desconexiones físicas inmediatas a través del canal nativo connectionState.
- **Ejecución Web (kIsWeb):** En navegadores compatibles con Web Bluetooth, el escaneo requiere un gesto explícito del usuario (pulsación en Buscar dispositivos) por restricciones de seguridad del navegador, y las desconexiones abruptas se capturan mediante timeout de escritura GATT.

02 Arquitectura de Software y Enrutamiento

2.1 Organización modular del código fuente (lib/)

Directorio / Archivo	Rol arquitectónico	Clases y responsabilidades clave
`lib/screens/`	Capa de Presentación	LoginScreen, BluetoothScreen, ConfigEntrenamientoScreen, EntrenamientoActivoScreen, SettingsScreen.
`lib/provider/`	Lógica y Estado	bluetooth_provider.dart, sound_provider.dart, locale_provider.dart, theme_provider.dart, user_mode_provider.dart, perfil_provider.dart.

Directorio / Archivo	Rol arquitectónico	Clases y responsabilidades clave
`lib/l10n/`	Localización	AppLocalizations: diccionario centralizado de cadenas traducidas y formateadores dinámicos.
`assets/sounds/`	Recursos Estáticos	Archivos de audio inicio.mp3 y fin.mp3 empaquetados en el bundle de la aplicación.

2.2 Gestión de estado reactiva (Riverpod)

Las vistas heredan de ConsumerStatefulWidget para suscribirse de forma selectiva a los proveedores sin acoplar la lógica de negocio a los widgets:

- **bluetoothProvider:** Expone el estado BluetoothState (conectado, device, ultimoTiempo, reactionTicket, desconexionInesperadaTicket) y los métodos conectar(), desconectar() y enviarComando().
- **Suscripción sin reconstrucción (ref.listen):** En EntrenamientoActivoScreen.build(), ref.listen<BluetoothState> observa cambios de estado sin reconstruir el árbol visual: detecta transiciones de conectado == true a false para disparar la alerta de desconexión, e incrementa reactionTicket para procesar cada toque del sensor.
- **Proveedores de configuración:** Gestionan de forma independiente el tema visual (ThemeMode), el idioma activo (Locale), el interruptor de Modo Profesional (bool), el perfil activo seleccionado y el estado global de audio.

2.3 Mapa de rutas (GoRouter)

Ruta	Pantalla destino	Método y parámetros
`/login`	LoginScreen	context.go('/login') — Pantalla inicial de autenticación y recuperación de clave.
`/register`	RegisterScreen	context.go('/register') — Alta de nuevos usuarios en Firebase Auth.
`/home`	HomeScreen	context.go('/home') — Panel principal tras iniciar sesión o finalizar una rutina.
`/bluetooth`	BluetoothScreen	context.push('/bluetooth') — Escaneo y vinculación con la central ESP32.
`/entrenamiento-activo`	EntrenamientoActivoScreen	context.push('/entrenamiento-activo', extra: config) — Recibe Map<String, dynamic>.
`/edit-profile`	EditProfileScreen	context.push('/edit-profile') — Edición de nombre, avatar y fecha de nacimiento.
`/terminos-privacidad`	TerminosPrivacidadScreen	context.push('/terminos-privacidad') — Documentación legal accesible desde login y ajustes.

03 Protocolo BLE y Comunicación con Hardware (ESP32)

3.1 Descubrimiento y filtrado por Service UUID

Para evitar listar dispositivos Bluetooth ajenos al sistema (auriculares, relojes o televisores), el método _startScan() filtra los paquetes de advertising mediante un UUID de servicio exclusivo de 128 bits y limita la ventana de búsqueda a 8 segundos:

Configuración de escaneo BLE (bluetooth_screen.dart)

```
await FlutterBluePlus.startScan(
  withServices: [Guid("4fa8691a-0393-40a3-974f-0759e09f114d")],
  timeout: const Duration(seconds: 8),
);
```

3.2 Protocolo de comandos TX (App → Central ESP32)

Una vez descubiertos los servicios GATT, la aplicación escribe tramas de texto codificadas en UTF-8 sobre la característica de recepción del microcontrolador (state.characteristicRx) con un tiempo límite de escritura de 3 segundos:

Operación	Formato de trama UTF-8	Ejemplo y descripción
Encender Pod LED	EMPEZAR:<podIndex>,<R>,<G>,<B>	EMPEZAR:0,0,255,0 → Enciende el Pod 1 (índice base 0) en color Verde al 100% de intensidad.
Apagar Pod LED	EMPEZAR:<podIndex>,0,0,0	EMPEZAR:2,0,0,0 → Apaga los LEDs del Pod 3 (índice 2) enviando valores RGB en cero.

Antes de construir la trama, obtenerRGBRonda() aplica el factor de brillo configurado (intensidad entre 10% y 100%) multiplicando cada canal base por (intensidad / 100.0) y redondeando al entero más cercano entre 0 y 255.

3.3 Recepción de eventos RX y tolerancia a fallos

- **Notificación de toque (reactionTicket):** Cuando el sensor de proximidad/toque de un pod es activado, el ESP32 notifica el tiempo transcurrido (en segundos). BluetoothProvider actualiza ultimoTiempo e incrementa el contador reactionTicket, disparando procesarTiempoRespuesta() en la pantalla activa.
- **Detección de desconexión abrupta:** El sistema combina la suscripción al stream connectionState del dispositivo con un temporizador watchdog y captura de errores en characteristicRx.write(). Si se pierde la conexión durante el entrenamiento, se invoca desconectar(inesperada: true), se cancelan todos los Timer activos y se presenta un diálogo modal bloqueante (barrierDismissible: false) para retornar de forma segura a /home.

04 Motor de Entrenamiento y Temporizadores

4.1 Contrato de configuración (Map<String, dynamic> config)

ConfigEntrenamientoScreen valida previamente que exista una conexión BLE activa y que en Modo Bicolor los dos colores elegidos sean distintos. Luego empaqueta los parámetros de la sesión en un mapa enviado vía GoRouter:

Clave	Tipo Dart	Rango / Valores válidos	Descripción
`modo`	String	'rondas' | 'tiempo' | 'bicolor' | 'manual'	Define la máquina de estados del entrenamiento.
`cantidadPods`	int	1 a 6	Límite superior para la selección aleatoria Random().nextInt(cantidadPods).
`rondasTotal`	int	5 a 30 (0 en tiempo/manual)	Cantidad objetivo de rondas antes de llamar a finalizarEntrenamiento().
`tiempoLimiteSeg`	int	10 a 120	Duración total en segundos cuando modo == 'tiempo'.
`colorAleatorio`	bool	true | false	Si es true, selecciona un color al azar de listaRGB en cada ronda.
`rgbBase`	List<int>	[R, G, B] (0..255)	Color principal (Color 1 a tocar en modo bicolor).
`rgbBase2`	List<int>	[R, G, B] (0..255)	Color distractor (Color 2 a ignorar en modo bicolor).
`intensidad`	int	10 a 100	Porcentaje de escalado de brillo aplicado a los canales RGB.
`tiempoEntrePodsSeg`	double	0.2 a 5.0	Retardo (Future.delayed) entre rondas consecutivas.
`tiempoDistractorSeg`	double	1.0 a 5.0	Ventana de auto-apagado cuando aparece el Color 2 en modo bicolor.

4.2 Gestión de temporizadores y ciclo de vida

EntrenamientoActivoScreen coordina hasta tres instancias independientes de Timer (dart:async):

- **_timerCuentaRegresiva:** Temporizador periódico de 1 segundo que decrementa cuentaRegresiva de 3 a 1. Al iniciar dispara playStart() y al llegar a 0 habilita el cronómetro (si corresponde) y lanza iniciarSiguienteRonda().
- **_timerTiempo:** Temporizador periódico de 1 segundo activo exclusivamente en modo == 'tiempo'; al llegar a 0 invoca finalizarEntrenamiento().

- **_timerDistractor:** Temporizador de disparo único (one-shot) utilizado en modo == 'bicolor' cuando esColor1Actual == false. Si el usuario no toca el pod durante tiempoDistractorSeg, apaga el LED sin sumar error y avanza a la siguiente ronda.

> **Prevención de fugas de memoria y apagado de seguridad**
> Tanto en el método dispose() del State como en el callback onPopInvokedWithResult de PopScope, la pantalla cancela explícitamente los tres temporizadores y ejecuta _apagarPodActual(actualizarUI: false) para garantizar que ningún LED quede encendido en el hardware si el usuario abandona la pantalla.

05 Subsistema de Audio e Internacionalización

5.1 Servicio de audio (SoundService)

Implementado en lib/provider/sound_provider.dart mediante audioplayers, encapsula una única instancia reutilizable de AudioPlayer configurada en ReleaseMode.stop. Incorpora dos mecanismos de control:

- **Silenciado global:** Consulta soundEnabledProvider antes de cualquier reproducción; si está en false, retorna inmediatamente sin acceder al hardware de audio.
- **Filtro anti-rebote (Debounce 600 ms):** Registra _ultimoArchivo y _ultimoPlay (DateTime) para descartar llamadas duplicadas sobre el mismo archivo dentro de una ventana de 600 milisegundos, evitando solapamientos por doble pulsación.

5.2 Internacionalización y soporte bidireccional (l10n)

La aplicación soporta 10 idiomas gestionados reactivamente por localeProvider: Español (es), Inglés (en), Portugués (pt), Italiano (it), Francés (fr), Alemán (de), Árabe (ar), Hebreo (he), Japonés (ja) y Chino (zh). La interfaz utiliza alineaciones direccionales (AlignmentDirectional.centerEnd) para adaptarse automáticamente a idiomas de lectura de derecha a izquierda (RTL) como Árabe y Hebreo, y sincroniza FirebaseAuth.instance.setLanguageCode(langCode) al enviar correos de restablecimiento.

06 Autenticación y Modelo de Datos en Firebase

6.1 Flujo de autenticación (Firebase Auth)

- **Login:** Inicio de sesión mediante signInWithEmailAndPassword() con mapeo de códigos de excepción (invalid-credential, wrong-password, user-not-found, invalid-email) a mensajes localizados.
- **Restablecimiento de clave:** Recuperación de contraseña integrada en LoginScreen mediante sendPasswordResetEmail(email) previa configuración del código de idioma activo.
- **Logout:** Cierre de sesión limpio mediante FirebaseAuth.instance.signOut() seguido de redirección con context.go('/login').

6.2 Esquema JSON en Firebase Realtime Database

Los datos se estructuran bajo el nodo raíz usuarios/{uid}, donde {uid} corresponde al identificador único asignado por Firebase Authentication:

Estructura jerárquica en Realtime Database

```
usuarios/
 └── {uid}/
      ├── nombre: "Benjamin"
      ├── photoUrl: "..."   // URL o imagen codificada en Base64
      ├── fecha_nacimiento: "2004-08-15"       // Formato ISO-8601 (detección de cumpleaños)
      └── sesiones/
           └── {pushId}/
                ├── rondas: 10
                ├── modo: "rondas"
                ├── cantidad_pods: 4
                ├── promedio_seg: 0.482
                ├── tiempos: [0.512, 0.460, ...]
                ├── errores: 0
                ├── perfil_id: "principal"
                ├── perfil_nombre: "Uso Personal"
                ├── notas_sesion: "Desplazamiento lateral"
                └── timestamp: 1731500000000      // ServerValue.timestamp
```

Campo en sesiones/{pushId}	Tipo	Regla de negocio / Observación
`rondas`	int	Cantidad total de toques válidos registrados (tiemposObtenidos.length).
`promedio_seg`	double	Media aritmética de tiempos; vale 0.0 si no hubo toques válidos.
`tiempos`	List<double>	Arreglo ordenado cronológicamente con cada tiempo de reacción en segundos.
`errores`	int	Contador de toques sobre el Color 2 en modo bicolor (0 en otros modos).
`perfil_id / perfil_nombre`	String	Identifica al deportista en Modo Profesional o toma 'principal' / 'Uso Personal'.
`timestamp`	ServerValue	Marca de tiempo asignada por el servidor de Firebase; escritura con timeout de 6s.

07 Entorno de Desarrollo, Compilación y Depuración

7.1 Requisitos del entorno y permisos Android

- **Herramientas de compilación:** Requiere Flutter SDK instalado, Android SDK Command-line Tools configurados y licencias aceptadas (flutter doctor --android-licenses).
- **Permisos nativos BLE:** Para operar BLE en Android 12+ y versiones anteriores, el manifiesto y el sistema operativo requieren permisos de escaneo/conexión Bluetooth y ubicación precisa (ACCESS_FINE_LOCATION) junto con el servicio de Ubicación (GPS) encendido en el dispositivo.

7.2 Consideraciones de depuración en VS Code (Web Bluetooth)

Comportamiento del depurador ante desconexiones simuladas en Chrome

Al depurar la versión Web en VS Code (F5), si se apaga físicamente el ESP32 durante una sesión, el motor de Web Bluetooth lanza una excepción asíncrona en characteristicRx.write() que ya está controlada dentro de un bloque try/catch para ejecutar desconectar(inesperada: true). Si VS Code tiene activados los puntos de interrupción en All Exceptions o Uncaught Exceptions, el depurador pausará la ejecución en esa línea. Para probar el flujo completo sin interrupciones del IDE, desmarcar ambos checkboxes en la vista Run and Debug → Breakpoints o ejecutar con Ctrl + F5 (Run Without Debugging).

7.3 Comandos habituales de ejecución y empaquetado

Comandos CLI de Flutter

```
# Verificar estado del entorno, SDK de Android y dispositivos conectados
flutter doctor -v

# Ejecutar en tablet/teléfono Android conectado por depuración USB
flutter run

# Compilar paquete instalable APK en modo Release
flutter build apk --release
```